Bynry Backend Engineering Intern- Case Study

Name : Mahesh Kumbhar

Part 1: Code Review & Debugging

1. Issues Identified

- No input validation (missing/null values)
- SKU uniqueness not checked
- Two separate DB commits (not atomic)
- No error handling (if failure occurs)
- Product may exist in multiple warehouses but logic assumes single
- Price not validated (decimal/negative)
- Inventory created without checking existing record

2. Impact in Production

- Duplicate SKUs → data inconsistency
- Partial data saved → product created but inventory missing
- App crashes if data missing
- Incorrect stock tracking
- Security issues (invalid input)

3. Fixed Version (Logic Explanation)

I am not very familiar with Flask/Python, so I am explaining my solution in simple logic:

Improved Approach:

1. Validate all required fields (name, sku, price)

2. Check if SKU already exists

3. Start a database transaction

4. Create product
5. Create inventory entry
6. Commit both together
7. If any error occurs, rollback everything

Pseudo Code:

```
Get request data
If name or sku is missing:
    return error
Check if SKU already exists:
    If yes → return error
Start transaction
Create product
Create inventory for that product
If everything is successful:
    Commit transaction
Else:
    Rollback transaction

Return success response
```

Part 2: Database Design

1. Tables Design

Companies

- id (Primary Key)
- name

Warehouses

- id (Primary Key)
- company_id (Foreign Key)
- name

Products

- id (Primary Key)
- name
- sku (Unique)
- price

Inventory

- id (Primary Key)
- product_id (Foreign Key)
- warehouse_id (Foreign Key)
- quantity

Suppliers

- id (Primary Key)

- name
- contact_email

Product_Suppliers
- product_id (Foreign Key)
- supplier_id (Foreign Key)

Inventory_Logs
- id
- product_id
- warehouse_id
- change
- timestamp

---

2. Missing Requirements (Questions)

1. What defines “recent sales activity”?
2. Can one product have multiple suppliers?
3. How are bundle products stored?
4. Is threshold same for all products or different?
5. Can inventory go negative?

3. Design Decisions

- SKU is unique to avoid duplication
- Separate inventory table to support multiple warehouses

- Logs table helps track stock changes
- Foreign keys maintain relationships
- Indexing can be applied on SKU and product_id for performance

Part 3: API Implementation

Endpoint

GET /api/companies/{company_id}/alerts/low-stock

1. Approach

Since I am not familiar with Flask/Node.js, I am explaining my logic:

1. Get company_id from request
2. Fetch all warehouses of that company
3. Fetch all inventory records for those warehouses
4. For each product:
   - Check if quantity is less than threshold
   - If yes, add it to alert list
5. Attach supplier information
6. Return response with alerts and total count

2. Pseudo Code

Get company_id

Fetch warehouses of company

```
For each warehouse:
    Get inventory

For each inventory item:
    If quantity < threshold:
        Add to alerts list

Return:
    alerts list
    total alerts count
```

3. Edge Cases

- No warehouses found
- No inventory data
- Product has no supplier
- Stock is zero or negative
- Threshold value missing

4. Assumptions

- Default threshold is 20
- One main supplier per product
- Recent sales logic is not defined, so assumed basic filtering